

Hierarchical Imitation Learning for Stochastic Environments

Maximilian Igl*, Punit Shah*, Paul Mougin*, Sirish Srinivasan*, Tarun Gupta†, Brandyn White*,
Kyriacos Shiarlis*, Shimon Whiteson*

Abstract—Many applications of imitation learning require the agent to generate the full distribution of behaviour observed in the training data. For example, to evaluate the safety of autonomous vehicles in simulation, accurate and diverse behaviour models of other road users are paramount. Existing methods that improve this *distributional realism* typically rely on hierarchical policies. These condition the policy on *types* such as goals or personas that give rise to multi-modal behaviour. However, such methods are often inappropriate for stochastic environments where the agent must also react to external factors: because agent types are inferred from the observed future trajectory during training, these environments require that the contributions of internal and external factors to the agent behaviour are disentangled and only internal factors, i.e., those under the agent’s control, are encoded in the type. Encoding future information about external factors leads to inappropriate agent reactions during testing, when the future is unknown and types must be drawn independently from the actual future. We formalize this challenge as distribution shift in the conditional distribution of agent types under environmental stochasticity. We propose *Robust Type Conditioning* (RTC), which eliminates this shift with adversarial training under randomly sampled types. Experiments on two domains, including the large-scale *Waymo Open Motion Dataset*, show improved distributional realism while maintaining or improving task performance compared to state-of-the-art baselines.

I. INTRODUCTION

Learning to imitate behaviour is crucial when reward design is infeasible [1, 2, 3, 4], for overcoming hard exploration problems [5, 6], and for realistic modelling of dynamical systems with multiple interacting agents [7]. Such systems, including games, driving simulations, and agent-based economic models, often have known state transition functions, but require accurate agent models to be realistic. For example, for driving simulations, which are crucial for accelerating the development of autonomous vehicles [8, 9], faithful reactions of all road users are paramount. Furthermore, it is not enough to mimic a single mode in the data; instead, agents must reproduce the full distribution of behaviours to avoid sim2real gaps in modelled systems [10, 11].

Current imitation learning (IL) methods fall short of achieving such *distributional realism*: while they are capable of generating individual trajectories that are realistic, they fail to match the full distribution of observed behaviour. Indeed, the adversarial training objective which enables state-of-the-art performance of most current IL methods is known to be prone to mode dropping in practice [12, 13, 14], even though it optimises a distribution-matching objective in principle [15, 16, 17]. Furthermore, progress on distributional realism is hindered by a lack of suitable benchmarks, with most relying on

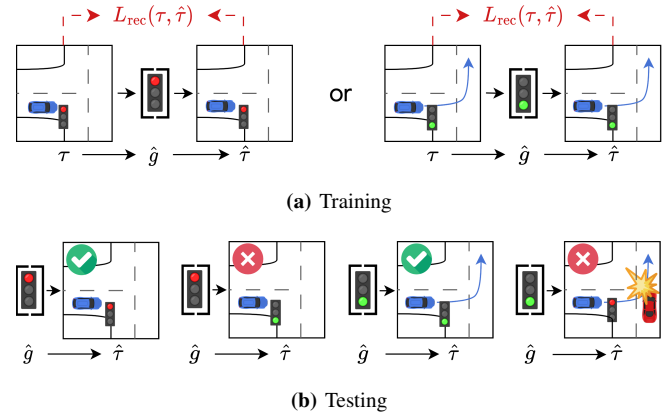


Fig. 1: Example highlighting how stochastic environments can cause out-of-distribution issues for hierarchical policies. **Top:** During training, the latent $\hat{g}$ is inferred from the future trajectory τ in the data using an encoder $e_{\theta}(\hat{g}|\tau)$. In this example, it captures the driving direction and whether the light turns green. The policy $\pi_{\theta}(\hat{a}|\hat{g}, s)$ ‘decodes’ $\hat{g}$ by acting in the environment to generate $\hat{\tau}$. The reconstruction loss $\mathcal{L}_{\text{rec}}$ penalises differences between τ and $\hat{\tau}$, training the policy to follow $\hat{g}$. **Bottom:** During testing, without access to the future, the latent $\hat{g}$ must be sampled randomly from a prior $p_{\theta}(\hat{g})$ which was trained to match the *marginal* distribution of possible latents, i.e., it randomly samples red or green lights and possible driving directions. This can cause issues such as collisions when the random latent and the environment do not match. Because the prior cannot know the future, it might sample a red light while the real traffic light turns green (2nd example) or, worse, it might wrongly sample a green light, possibly leading to collisions (last example) if the agent follows the latent $\hat{g}$ as it was trained to do. On the other hand, random sampling of agent-internal decisions such as driving directions is unproblematic as these do not make assumptions about the future environment.

unimodal data and only evaluating task performance as measured by rewards, but not mode coverage or recall. By contrast, many applications, such as agent modeling for autonomous vehicles, require distributional realism in addition to good task performance. Consequently, our goal is to improve distributional realism while maintaining strong task performance.

To mitigate mode collapse and improve distributional realism in complex environments, previous work uses hierarchical policies in an autoencoder-like framework [12, 8, 9, 18]. During training, an encoder infers goals from observed future trajectories and the agent, conditioned on those goals, strives to imitate the original trajectory. At test time, a prior distribution proposes distributionally realistic goals, without requiring access to privileged future information. We refer to these goals as an agent’s inferred *type* since it can express not only goals, but many agent characteristics responsible for

* Waymo Research, † U. of Oxford. Work performed during internship.

multi-modal behaviour, such as persona, goal, or strategy.

However, as we show in section III, using such hierarchical policies in stochastic environments can create a distribution shift between training and testing, possibly leading to out-of-distribution inputs and reduced performance. Unfortunately, the autoencoder training does not prevent extrinsic information from being encoded. Intuitively, the type should only capture agent-intrinsic choices that are under the agent's control.

Consider a car waiting at an intersection (see fig. 1). During training, because the agent's type (e.g., goal and driving style) are not directly observed, they must be inferred from its future trajectory. However, this inferred type might not only capture their goal and driving style, but also external factors out of the agent's control, such as the time until the traffic light turns green. Even innocuous seeming type representations can leak external information; for example, a goal location extracted from the future trajectory can leak information about waiting times based on its distance to the starting position.

Capturing information about external events in the inferred type causes problems at test time, when the type must be sampled randomly without foresight of future events. For example, if the type contains information about traffic light timings, the actual timing on the test data will almost surely differ from the randomly sampled one, resulting in out-of-distribution inputs to the policy which never encountered such a mismatch during training where the type was always inferred from the actual future. Furthermore, the agent might have learned to ignore the actual traffic light, instead relying entirely on the inferred type that was always optimal during training. This could cause it to enter the intersection too early, resulting in potentially catastrophic consequences such as collisions.

Existing hierarchical work either assumes no external stochasticity in the environment [12, 8], relies on manually designed type representations that cannot capture external events but limit expressiveness [9], or relies on manually designed cost functions and type filters that mitigate the performance degradation but do not solve the underlying problem and induce biases in the learned behaviour [18].

In this paper, we identify the challenges arising under stochastic environments and formulate them as a new form of distribution shift for hierarchical policies. Unlike the familiar covariate shift in the state distribution [19], this *conditional type shift* occurs in the distribution of the inferred latent type. It greatly reduces performance by yielding causally confused agents that rely on the latent type for information about external factors, instead of inferring them from the latest environment observation. We propose *Robust Type Conditioning* (RTC) to eliminate this distribution shift through a coupled adversarial training objective under randomly sampled types. We do not require access to an expert, counterfactuals, or manually specified type labels for trajectories.

Experimentally, we show the need for improved distributional realism in state-of-the-art imitation learning techniques such as GAIL [16]. Furthermore, we show that naively trained hierarchical models with inferred

types improve distributional realism, but exhibit poor task performance in stochastic environments. By contrast, RTC can maintain good task performance in stochastic environments while improving distributional realism. We evaluate RTC on the illustrative *Double Goal Problem* as well as the large scale *Waymo Open Motion Dataset* [20] of real driving behaviour.

II. BACKGROUND

We are given a dataset $\mathcal{D} = \{\tau_i\}_{i=1}^N$ of N trajectories $\tau_i = s_0^{(i)}, a_0^{(i)}, \dots, s_T^{(i)}$, drawn from $p(\tau)$ of one or more experts interacting with a stochastic environment $p(s_{t+1}|s_t, a_t)$ where $s_t \in \mathcal{S}$ are states and $a_t \in \mathcal{A}$ are actions. Our goal is to learn a policy $\pi_\theta(a_t|s_t)$ to match $p(\tau)$ when replacing the unknown expert and generating rollouts $\hat{\tau} \sim p(\hat{\tau}) = p(s_0) \prod_{t=0}^{T-1} \pi_\theta(\hat{a}_t|\hat{s}_t)p(\hat{s}_{t+1}|\hat{s}_t, \hat{a}_t)$ from the initial states $s_0 \sim p(s_0)$. We simplify notation and write $\hat{\tau} \sim \pi_\theta(\hat{\tau})$ and $\tau \sim \mathcal{D}(\tau)$ to indicate rollouts generated by the policy or drawn from the data respectively. Expectations $\mathbb{E}_{\tau \sim \mathcal{D}}$ and $\mathbb{E}_{\hat{\tau} \sim \pi_\theta}$ are taken over all pairs $(s_t, a_t) \in \tau$ and $(\hat{s}_t, \hat{a}_t) \in \hat{\tau}$.

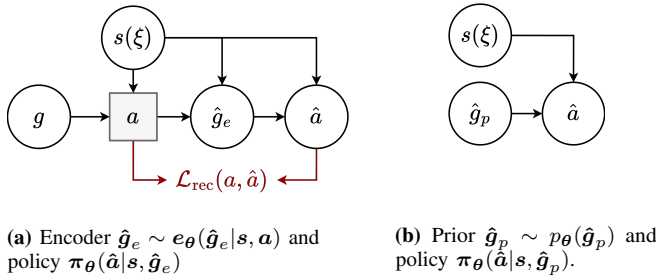
Previous work [e.g., 19, 16] shows that a core challenge of learning from demonstration is reducing or eliminating the covariate shift in the state-visitation frequencies $p(s)$ caused by accumulating errors when using π_θ . Unfortunately, *Behavioural Cloning* (BC), a simple supervised training objective optimising $\max_\theta \mathbb{E}_{\tau \sim \mathcal{D}} [\log \pi_\theta(a_t|s_t)]$ is not robust to it. To overcome covariate shift, generative adversarial imitation learning (GAIL) [16] optimises π_θ to fool a learned discriminator $D_\phi(\hat{a}_t, \hat{s}_t)$ that is trained to distinguish between trajectories in $\mathcal{D}$ and those generated by π_θ :

$$\min_{\theta} \max_{\phi} \mathbb{E}_{\hat{\tau} \sim \pi_\theta} \left[\log(D_\phi(\hat{a}_t, \hat{s}_t)) \right] + \quad (1)$$

$$\mathbb{E}_{\tau \sim \mathcal{D}} \left[\log(1 - D_\phi(a_t, s_t)) \right]. \quad (2)$$

The policy can be optimised using reinforcement learning, by treating the log-discriminator scores as costs, $r_t = -\log D_\phi(\hat{a}_t, \hat{s}_t)$. Alternatively, if the policy can be reparameterized [21] and the environment is differentiable, the sum of log discriminator scores can be optimised directly without relying on high-variance score function estimators by backpropagating through the transition dynamics, $\mathcal{L}_{\text{adv}}(\hat{\tau}) = \mathbb{E}_{\hat{\tau} \sim \pi_\theta} [\sum_t -\log D_\phi(\hat{a}_t, \hat{s}_t)]$. We refer to this as *Model-based GAIL* (MGAIL) and assume a known differentiable environment instead of a learned model as in [17].

In this work, we are concerned with multimodal distributions $p(\tau)$ and how mode collapse can be avoided when learning π_θ . To this end, we assume the dataset is sampled from $p(\tau) = p(s_0) \int p(g)p(\xi) \prod_{t=0}^T p(a_t|s_t, g)p(s_{t+1}|s_t, a_t, \xi) d\xi dg$, where g is the agent *type*, expressing agent characteristics such as persona, goal, or, strategy, and ξ is a random variable capturing the stochasticity in the environment, i.e., $p(s_{t+1}|s_t, a_t, \xi)$ is a delta distribution $\delta_{f(s_t, a_t, \xi)}(s_{t+1})$ for some transition function f . We call an agent realistic if its generated trajectories $\hat{\tau} \sim \pi_\theta$ lie in the support of $p(\tau)$. We call an agent *distributionally realistic* if its distribution over trajectories matches the data, i.e. $p(\hat{\tau}) \approx p(\tau)$. As we show



$P_{\mathcal{D}}(s, a)$		
$\begin{matrix} a(s, g) \\ s(\xi) \end{matrix}$	a_0	a_1
s_0	$1 - \epsilon$	ϵ
s_1	ϵ	$1 - \epsilon$

$\xi \sim \mathcal{B}(0.5), g \sim \mathcal{B}(1 - \epsilon)$
 $s(\xi) = \xi s_0 + (1 - \xi) s_1$
 $a(s_i, g) = g a_i + (1 - g) a_j$

(c) Example dataset.

Fig. 2: Simplified, non-temporal setup with environmental noise ξ and unobserved true agent type g . The inferred type $\hat{g}$ is sampled from $e_\theta(\hat{g}_e|s, a)$ during training (top-left) and $p_\theta(\hat{g}_p)$ otherwise (top-right). The control policy is $\pi_\theta(\hat{a}|s, \hat{g})$. Circles are random variables and squares deterministic functions. The loss $\mathcal{L}(a, \hat{a})$ penalises differences between a and $\hat{a}$. Bottom: Example data, $\mathcal{B}$ denotes Bernoulli distributions.

in section VI, current non-hierarchical adversarial methods [16] are not distributionally realistic.

To combat mode collapse, hierarchical methods [e.g., 12, 22, 8, 9, 18] often rely on an encoder to infer latent agent types $\hat{g}_e$ from trajectories during training, $\hat{g}_e \sim e_\theta(\hat{g}_e|\tau)$, and optimise the control policy $\pi_\theta(\hat{a}_t|\hat{s}_t, \hat{g}_e)$ to generate trajectories $\hat{\tau}_e$ similar to τ : $\hat{\tau}_e \sim p(\hat{\tau}_e|\hat{g}_e) = p(s_0) \prod_{t=0}^{T-1} \pi_\theta(\hat{a}_t|\hat{s}_t, \hat{g}_e) p(\hat{s}_{t+1}|\hat{a}_t, \hat{s}_t)$. As ground truth trajectories are not accessible during testing, a prior $p_\theta(\hat{g}_p)$, which has been trained to match the marginal distribution $p_e(\hat{g}_e) = \mathbb{E}_\tau [e_\theta(\hat{g}_e|\tau)]$, is used to sample distributionally realistic types $\hat{g}_p$. We indicate by subscript $\hat{g}_p$ or $\hat{g}_e$ whether the inferred type and trajectory are drawn from the prior distribution $p_\theta(\hat{g}_p)$ or encoder $e_\theta(\hat{g}_e|\tau)$. Subscripts are omitted for states and actions to simplify notation. Inferred types and predicted trajectories without subscripts indicate that either sampling distribution could be used. For information theoretic quantities we use capital letters $S, A, \hat{A}, \hat{G}$ and Ξ to denote the random variables for values $s, a, \hat{a}, \hat{g}$ and ξ .

III. CONDITIONAL TYPE SHIFT

Here we outline the challenge of *conditional type shift* that arises for hierarchical policies in stochastic environments. We provide a simple example illustrating the challenge and how it can be overcome, as well as formulate a proof for the exact conditions under which such a distribution shift occurs. These insights motivate the algorithm in section IV.

A. Simplified model

We use the simplified model in fig. 2. For intuition, we connect it to the example mentioned in the introduction of an agent approaching a traffic light. This model has two sources of randomness in the training data $\mathcal{D}$: the environmental noise

ξ (whether the traffic light is red or green) and the type g of the expert we are mimicking (whether the expert is paying attention). The crucial difference between g and ξ is that ξ represents *external* factors outside the agent's control to which it must react, while g encodes agent-internal decisions that can be taken independently of ξ . In this simple model, the temporal dimension is removed and the state s is a deterministic function of only ξ and not influenced by g . Hence, in this section we use s and ξ interchangeably.

During training, the inferred type $\hat{g}_e$ is drawn from the encoder $e_\theta(\hat{g}_e|\tau)$ which has access to the future 'trajectories' $\tau = (s, a)$ in the data. During testing, without access to τ , a prior $p_\theta(\hat{g}_p)$ is used to sample $\hat{g}_p$. Actions $\hat{a}$ are drawn from the learned control policy $\pi_\theta(\hat{a}|s, \hat{g})$ and a reconstruction loss $\mathcal{L}_{\text{rec}}(a, \hat{a})$ is minimised. As typical in autoencoders, the prior $p_\theta(\hat{g}_p)$ is trained to match the *marginal* distribution of the encoder $p_e(\hat{g}_e) = \mathbb{E}_\tau [e_\theta(\hat{g}_e|\tau)]$ by minimizing $\mathcal{L}_{\text{kl}}(\tau) = \mathbb{E}_{\tau \sim \mathcal{D}} [\text{KL}[p_\theta(\hat{g}_p) \| e_\theta(\hat{g}_e|\tau)]]$.

B. Only external factors of influence

We first describe a scenario with only external sources of stochasticity that serves as a minimal example of how things can go wrong due to conditional type shift. As there are no agent-internal decisions, hierarchies are unnecessary in this minimal scenario. In section III-C, we extend this example to include agent-internal decisions which hierarchical policies capture well.

Consider the example data in fig. 2c with $\epsilon = 0$, i.e., for now we assume the expert is always paying attention. The environment can be in two states. Half the time, it is in s_0 , where the traffic light is red and the agent always takes action $a_0 = \text{stop}$. Otherwise, in s_1 , the traffic light is green and the agent takes action $a_1 = \text{go}$.

During training, the encoder observes the actual future $\tau = (s, a)$ in the data and proposes the type $\hat{g}_e \sim e_\theta(\hat{g}_e|\tau)$ with $\hat{g}_e \in \{0, 1\}$. This allows, for example, the following **solution 1** which minimises the reconstruction loss $\mathcal{L}_{\text{rec}}(a, \hat{a})$:

$$\hat{g}_e(s_i, a_j) = j \quad \text{and} \quad \pi_\theta(\hat{a}|s_i, \hat{g}_e) = a_{\hat{g}_e}$$

The encoder encodes the desired action in the type $\hat{g}_e$ and the policy follows $\hat{g}_e$ while ignoring s . This constitutes a perfect solution during training. However, during testing, we do not have access to τ and must instead draw types randomly from the prior $\hat{g}_p \sim p_\theta(\hat{g}_p)$, which matches the *marginal* distribution of the encoder, i.e., $p_\theta(\hat{g}_p) = p_e(\hat{g}_e) = \mathcal{B}(0.5)$. Here, $\mathcal{B}$ is the Bernoulli distribution and the prior is drawing types $\hat{g}_p \in \{0, 1\}$ with equal probability because it cannot know the stochastic environment state s in advance.

The conditional type shift arises because this marginal distribution does not need to match the *conditional* type distribution in specific states, i.e., $p_\theta(\hat{g}_p) \neq e_\theta(\hat{g}_e|s, a)$. For example, the prior might sample $\hat{g}_p = 1$ while the stochastic environment shows a red light ($s = s_0$). The resulting input to the policy, $(s_0, \hat{g} = 1)$, was never seen during training where state and type always matched, i.e., the input pairs were either $(s_0, \hat{g} = 0)$ or $(s_1, \hat{g} = 1)$.

If the policy generalises to this new input by following the type, as was optimal during training, it randomly stops or goes, clearly not reproducing the data distribution and causing potentially catastrophic mistakes such as collisions.

This problem always occurs when information about *external* stochastic factors is captured by the type. As there are no internal decision by the agent in this example, the ideal solution is for the type to not encode any information. In section III-C we show that the conditional type shift problem does not arise when only agent-internal decisions are encoded, as these can be taken by the agent independently from the environment stochasticity.

C. External and internal factors of influence

To express this, we now introduce $\epsilon > 0$ as the probability that the agent decides not to pay attention to the traffic light. Hence, the expert now either follows the traffic light with $p(a_i|s_i) = 1 - \epsilon$, or ignores it with $p(a_{\neq i}|s_i) = \epsilon$.

The previous *solution 1* is still viable during training, minimising the reconstruction loss, but still fails during testing as it generates an action distribution which ignores the traffic light 50% of the time, i.e., $p(\hat{a}_{\neq i}|s_i) = 0.5$, in contrast to the expert, which only deviates with probability $p(a_{\neq i}|s_i) = \epsilon$.

By contrast, **solution 2** avoids the conditional type shift but successfully encodes the agent-internal decision:

$$\hat{g}_e(s_i, a_j) = \begin{cases} 0 & \text{if } i = j \\ 1 & \text{if } i \neq j \end{cases}, \quad \pi_\theta(\hat{a}|s_i, \hat{g}) = \begin{cases} a_i & \text{if } \hat{g} = 0 \\ a_{\neq i} & \text{if } \hat{g} = 1 \end{cases}$$

Here the latent type $\hat{g}$ only captures whether the agent pays attention ($\hat{g} = 0$) or not ($\hat{g} = 1$). Now the marginal encoder type distribution is $p_e(\hat{g}_e = 0) = 1 - \epsilon$ and hence we have for the learned prior $p_\theta(\hat{g}_p) = \mathcal{B}(\epsilon)$, correctly reproducing the data in all states.

To summarize, hierarchical policies in stochastic environments only generalise at test time when the type only conveys information about agent-internal features and is uncorrelated with any external stochastic events during training.

For simplicity, the model in fig. 2c has only one time-step. For temporally extended data, the states s_t depend not only on ξ , but also on g or $\hat{g}$, complicating theoretical treatment. Nevertheless, seeing ξ as all *future* stochasticity in the environment, the same challenges arise. In realistic driving scenarios ξ not only captures traffic lights, but also the reactions of other agents in the scene. Similarly, agent-internal factors include a wide range of information such as goals, driving-style or level of attention.

D. Theorem

Here we provide an information theoretic proof that agents that, during training, rely on the inferred type to acquire information about external events, do not react appropriately to environment stochasticity during testing. This formalises the previous discussion but is not needed to follow subsequent sections of the paper.

Theorem 1 *The hierarchical autoencoding model $p_\theta(\hat{a}|s, \alpha)$ and test policy $p_\theta(\hat{a}|s)$ are as described above,*

sampling latent types from encoder e_θ and prior p_θ respectively. We assume an optimal reconstruction loss $\mathcal{L}_{rec} = 0$ on the training data $P_D(s, \alpha)$. For the training distribution $P(s, \alpha, \hat{g}_e) = P_D(s, \alpha)e_\theta(\hat{g}_e|s, \alpha)$ and test-time distribution $P(s, \hat{a}, \hat{g}_p) = P_D(s)p_\theta(\hat{g}_p)\pi_\theta(\hat{a}|s, \hat{g}_p)$ we have that if $H(A|\hat{G}_e) < I(S, A)$ and $H(A|\hat{G}_e) = H(\hat{A}|\hat{G}_p)$, then $I(S, \hat{A}) < I(S, A)$ and consequently $H(\hat{A}|S) > H(A|S)$.

We denote by $H(X)$ the entropy, by $H(X|Y)$ the conditional entropy, by $I(X, Y)$ the mutual information and by $I(X, Y|Z)$ the conditional mutual information between random variables. Intuitively, $H(A|\hat{G}_e) < I(S, A)$ if the encoder captures information about A in $\hat{G}_e$ that is also accessible through S , i.e., information about external events. The condition $H(A|\hat{G}_e) = H(\hat{A}|\hat{G}_p)$ implies that the policy relies on this information in $\hat{G}$ to predict $\hat{A}$. If both conditions are true then $H(\hat{A}|S) > H(A|S)$, stating that the state S has less predictive power for the predicted action $\hat{A}$ than for actions A in the dataset, i.e.: that the policy is ignoring action-relevant information in the states.

Proof The proof relies on the *interaction information* $I(X, Y, Z)$, an extension of mutual information to three variables. Importantly, the interaction information can be positive or negative. A positive interaction information indicates that one variable explains some of the correlation between the other two while a negative interaction information indicates that one variable enhances their correlation.

The model $p_\theta(\hat{a}|s, \alpha) = e_\theta(\hat{g}_e|\alpha, s)\pi_\theta(\hat{a}|s, \hat{g}_e)$ is trained on the dataset $P_D(s, \alpha)$. Achieving minimal reconstruction loss is achieved only when $\hat{a} = \alpha$ is predicted with certainty, implying $H(\hat{A}|S, \hat{G}_e) = H(\hat{A}|S, \hat{G}_p) = 0$.

During training on the dataset $P_D(s, \alpha)$ the interaction information is positive because $H(A|\hat{G}_e) < I(S, A)$:

$$I(A, \hat{G}_e, S) = I(S, A) - I(S, A|\hat{G}_e) = I(S, A) - H(A|\hat{G}_e) + H(A|S, \hat{G}_e) > 0.$$

On the other hand, during testing, we have $I(\hat{G}_p, S) = 0$ because $\hat{G}_p$ is drawn independently of S . The interaction information becomes weakly negative:

$$I(\hat{A}, \hat{G}_p, S) = I(\hat{G}_p, S) - I(\hat{G}_p, S|\hat{A}) \leq 0$$

With $I(\hat{A}, \hat{G}_p, S) = I(S, \hat{A}) - H(\hat{A}|\hat{G}_p)$ we get

$$I(S, \hat{A}) - H(\hat{A}|\hat{G}_p) \leq 0 < I(S, A) - H(A|\hat{G}_e) \quad (3)$$

and hence, because by assumption $H(A|\hat{G}_e) = H(\hat{A}|\hat{G}_p)$, this gives us the desired result $I(S, \hat{A}) < I(S, A)$ from which $H(\hat{A}|S) > H(A|S)$ follows directly.

IV. ROBUST TYPE CONDITIONING

We present *Robust Type Conditioning* (RTC), a method for improving distributional realism in imitation learning while maintaining high task performance, even in stochastic environments. As shown in previous work [8, 9, 18], and confirmed in section VI, hierarchical policies trained in an autoencoder framework are currently the most effective approach at improving distributional realism. However, such policies require the latent type to be inferred from the

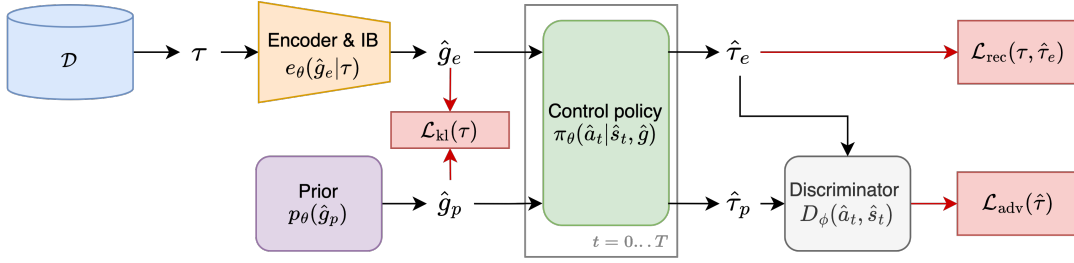


Fig. 3: Robust Type Conditioning (RTC): The control policy $\pi_{\theta}(\hat{a}_t|\hat{s}_t, \hat{g})$ is trained under inferred types $\hat{g}$ sampled from both the encoder $e_{\theta}(\hat{g}_e|\tau)$ and the prior $p_{\theta}(\hat{g}_p)$. The hierarchical loss $\mathcal{L}_{\text{vae}}(\tau, \hat{\tau}) = \mathcal{L}_{\text{rec}}(\tau, \hat{\tau}_e) + \beta \mathcal{L}_{\text{kl}}(\tau)$ improves distributional realism. The adversarial loss $\mathcal{L}_{\text{adv}}(\hat{\tau})$ under prior types prevents causally confused policies and ensures good task performance at test time, even in stochastic environments. $\mathcal{L}_{\text{kl}}(\tau)$ optimises the prior to sample distributionally realistic types.

future trajectory, which can cause problems in stochastic environments (see section III).

To overcome this limitation, we propose to combine the autoencoder training objective $\mathcal{L}_{\text{vae}} = \mathcal{L}_{\text{rec}} + \beta \mathcal{L}_{\text{kl}}$ with an additional adversarial objective $\mathcal{L}_{\text{adv}}$ utilising a learned discriminator $D_{\phi}(\mathbf{a}_t, \mathbf{s}_t)$. Importantly, this additional objective allows us to sample training types not only from the encoder, but also from the prior. When types are sampled from the prior, the hierarchical loss $\mathcal{L}_{\text{vae}}$ cannot be used as we generally do not have access to ground truth trajectories corresponding to this specific type, which are required for the reconstruction loss $\mathcal{L}_{\text{rec}}$. Instead, for these types, we only optimise the adversarial objective $\mathcal{L}_{\text{adv}}(\hat{\tau}) = \sum_t -\log D_{\phi}(\hat{\mathbf{a}}_t, \hat{\mathbf{s}}_t)$.

During training, we hence split each minibatch $\mathcal{B} = \{\tau^{(b)}\}_b^{N_b}$ of N_b trajectories sampled from $\mathcal{D}$ into two parts. For the fraction f of trajectories in $\mathcal{B}$ the rollouts $\hat{\tau}_e$ are generated from types sampled from the encoder $\hat{g}_e \sim e_{\theta}(\hat{g}_e|\tau)$ and objectives $\mathcal{L}_{\text{adv}} + \lambda \mathcal{L}_{\text{vae}}$ are optimised (first line in eq. (4)). For the remaining fraction $(1 - f)$ of trajectories, types are sampled from the prior $p_{\theta}(\hat{g}_p)$ and only $\mathcal{L}_{\text{adv}}$ is optimised (second line in eq. (4)).

Because the policy does not know whether the type is sampled from the encoder or prior, this combination ensures that policies follow agent-internal information in the type, due to the autoencoder training objective, but ignore any information in the type about external stochastic events, as this would lead to unrealistic trajectories under prior types, which are penalised by the adversarial training objective. Lastly, because the KL objective $\mathcal{L}_{\text{kl}}(\tau) = \mathbb{E}_{\tau \sim \mathcal{D}} [\text{KL}[p_{\theta}(\hat{g}_p) \| e_{\theta}(\hat{g}_e|\tau)]]$ minimises the amount of information encoded in the type, such unused information would not even be encoded.

Consequently, the full RTC loss is

$$\mathcal{L}_{\text{RTC}} = \mathbb{E}_{\mathcal{D}(\tau) e_{\theta}(\hat{g}_e|\tau) \pi_{\theta}(\hat{\tau}_e|\hat{g}_e)} [\lambda \mathcal{L}_{\text{adv}}(\hat{\tau}) + \mathcal{L}_{\text{vae}}(\tau, \hat{\tau})] + \mathbb{E}_{\mathcal{D}(\tau) p_{\theta}(\hat{g}_p) \pi_{\theta}(\hat{\tau}_p|\hat{g}_p)} [\lambda \mathcal{L}_{\text{adv}}(\hat{\tau})], \quad (4)$$

with

$$\begin{aligned} \mathcal{L}_{\text{vae}}(\tau, \hat{\tau}) &= \mathcal{L}_{\text{rec}}(\tau, \hat{\tau}) + \beta \mathcal{L}_{\text{kl}}(\tau) \\ \mathcal{L}_{\text{kl}}(\tau) &= \mathbb{E}_{\tau \sim \mathcal{D}} [\text{KL}[p_{\theta}(\hat{g}_p) \| e_{\theta}(\hat{g}_e|\tau)]] \\ \mathcal{L}_{\text{adv}}(\hat{\tau}) &= \sum_t -\log D_{\phi}(\hat{\mathbf{a}}_t, \hat{\mathbf{s}}_t), \end{aligned}$$

where $p_{\theta}(\hat{g}_p)$ is a learned prior, $e_{\theta}(\hat{g}_e|\tau)$ a learned trajectory encoder and $\pi_{\theta}(\hat{\tau}|\hat{g})$ is shorthand for generating trajectories

$\hat{\tau}$ by rolling out the learned control policy $\pi_{\theta}(\hat{a}|\hat{s}, \hat{g})$ in the environment. Parameters $\bar{\theta}$ are held fixed and λ and β are scalar weights. $D_{\phi}(\mathbf{a}_t, \mathbf{s}_t)$ is a learned per-timestep discriminator. Lastly, $\mathcal{L}_{\text{rec}}(\tau, \hat{\tau})$ is a reconstruction loss between τ and $\hat{\tau}_e$ which can take different forms. For example, in section VI-A we use the BC loss $\mathcal{L}_{\text{rec}}(\tau, \hat{\tau}) = -\log \pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t, \hat{g}_e)$ while in section VI-B we minimise the L_2 distance between agent positions in $\mathbf{s}_t$ and $\hat{\mathbf{s}}_t$. The loss $\mathcal{L}_{\text{kl}}(\tau)$ optimises the prior to propose *distributionally realistic* types by matching the marginal encoder distribution.

One can understand the problem of conditional type shift as one of *causally confused* policies which refer to the type for information about external stochastic events, instead of acquiring this information directly from the currently observed states. From this perspective, sampling from the prior constitutes a causal intervention $do(\hat{g})$ in which $\hat{g}$ is changed independently of the environmental factor ξ . [23] show that causal confusion can be avoided by applying such interventions and optimising the policy to correctly predict the counterfactual expert trajectory distribution, in our case $p_{\text{expert}}(\tau|\xi, do(\hat{g}))$. Unfortunately, we do not have access to this counterfactual trajectory. Instead, we rely on the generalisation of π_{θ} to get us ‘close’ to such a counterfactual trajectory for types $do(\hat{g})$ and then refine the policy locally using the adversarial objective.

We find that both continuous type representations with and discrete type representations using straight-through gradient estimation work well in practice (see section VI-B).

Optimisation of $\mathcal{L}_{\text{adv}}$ and $\mathcal{L}_{\text{rec}}$ can either be performed directly, similar to MGAIL [17], by using a differentiable environment and reparameterised policies and encoder [21] or by treating them as rewards and using RL methods such as TRPO [24, 16] or PPO [25]. The loss $\mathcal{L}_{\text{kl}}$ can always be optimised directly.

V. RELATED WORK

Hierarchical policies have been extensively studied in RL [e.g., 26, 27, 28, 29, 30] and IL. In RL, they improve exploration, sample efficiency and fast adaptation. By contrast, in IL, hierarchies are used to capture multimodal distributions, improve data efficiency [31, 32], and enable goal conditioning [33]. Similar to our work, [12] and [22] learn to encode trajectories into latent types that influence a control policy. Crucially, both only consider deterministic environments and

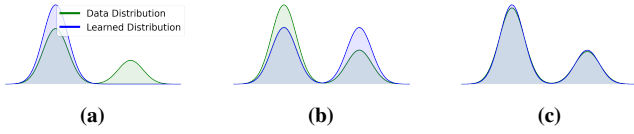


Fig. 4: Differences between *realism*, *coverage* and *distributional realism*. The data distribution $P(X_D)$ is shown in green, blue denotes a learned distribution $P_\theta(X_L)$. (a) Data from the learned distribution is *realistic*, i.e. $\text{supp}(X_L) \subseteq \text{supp}(X_D)$, but not *distributionally realistic*. (b) The learned distribution achieves *coverage* but not *distributional realism*: the frequencies of modes are not matched. (c) The learned distribution is *distributionally realistic*. In practice we measure distributional realism in selected features $h(X)$ as the dimensionality of $\mathcal{X}$ is too high.

hence avoid the distribution shifts and unwanted information leakage we address. They extend prior work in which the type, or context, is provided in the dataset [34], which is also assumed in [35]. [36] use a sampling method to infer latent types. [37] and [9] use manually designed encoders specific to road users by expressing future goals as sequences of lane segments. This avoids information leakage but cannot express all characteristics of human drivers, such as persona, and cannot transfer to other tasks. BITS [18] uses goal positions as types, which suffer from conditional type shift. Consequently, their method requires behaviour prediction and a manually specified cost function to filter goals that might mismatch with predicted futures. Futures states in deterministic environments [38], language [39], and predefined strategy statistics [40] have also been used as types.

Information theoretic regularization offers an alternative to learning hierarchical policies using the auto-encoder framework [41, 42]. However, these methods are less expressive since their prior distribution cannot be learned and only aim to cluster modes already captured by the agent but not penalize dropping modes in the data. This provides a useful inductive bias but often struggles in complex environments with high diversity, requiring manual feature engineering [43, 44].

Lastly, TrafficSim [8] uses IL to model driving agents, controlling all stochasticity in the scene but using independent prior distributions for separate agents. Hence, while the environment is assumed deterministic, conditional type shift can occur between the separate agent-types which are correlated during training but independent during testing. They use a biased “common sense” collision avoidance loss, motivated by covariate shift in visited states. Our work suggests that type shift might also explain the benefits gained. In contrast, our adversarial objective is unbiased.

VI. EXPERIMENTS

We show in two stochastic environments with multimodal expert behaviour that i) existing IL methods suffer from insufficient distributional realism, ii) hierarchical methods can suffer from conditional type shift and degrading task performance, and iii) RTC improves distributional realism while maintaining excellent task performance.

We compare the following models: MGAIL uses an adversarial training objective with learned discriminator. It also optimises a BC loss as we found this to improve performance.

Symphony [9] (called ‘MGAIL+H’ in the original paper), building on MGAIL, utilises future lane segments as manually specified types which avoid conditional type shift but limit expressiveness. InfoMGAIL [41] augments MGAIL to elicit distinct trajectories for different types by using an information-theoretic loss. This is an alternative training paradigm for hierarchical policies, besides using autoencoders with reconstruction loss. For fair comparison, our method RTC uses the same MGAIL implementation as adversarial objective. We investigate both continuous and discrete type representations, RTC-C and RTC-D. NaiveHierarchy is a hierarchical autoencoder not training on prior-sampled types (but also using the adversarial MGAIL objective) and hence experiencing conditional type shift and high collision frequency.

A. Double Goal Problem

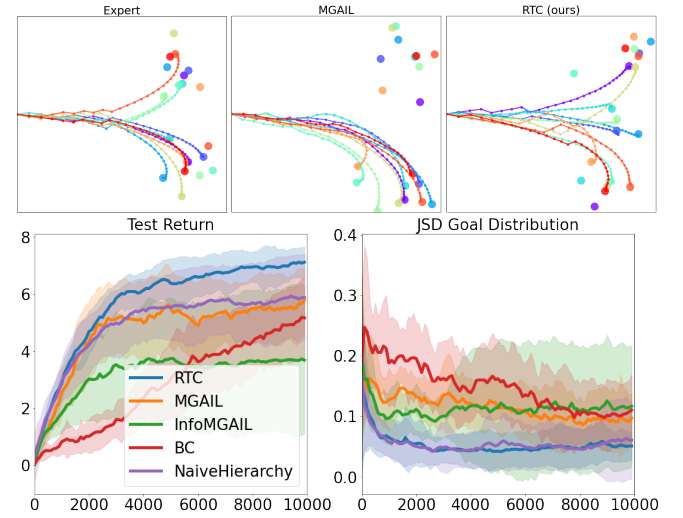


Fig. 5: *Top:* Visualization of ten randomly sampled goal pairs and associated trajectories. *Bottom:* Training curves, exponentially smoothed and averaged over 20 seeds. Shading shows the standard deviation. We show task performance as ‘Test Return’ (higher is better) and distributional realism as ‘JSD’ between the goal distribution of expert and agent (lower is better).

In the double goal problem, the expert starts from the origin and creates a multimodal trajectory distribution by randomly choosing and approaching one of two possible, slowly moving goals located on the 2D plane. Stochasticity is introduced through randomized initial goal locations and movement directions. Nevertheless, the *lower* and *upper* goal $\{g_l, g_u\}$ remain identifiable by their location as $y_l < 0$ for g_l and $y_u > 0$ for g_u (see fig. 5). While both goals are equally easy to reach, the expert has a preference $P(G = g_l) = 0.75$. Sufficiently complex expert trajectories prevent BC from achieving optimal performance, requiring more advanced approaches. The expert follows a curved path and randomly resamples the selected goal for the first ten steps to avoid a simple decision boundary along the x -axis in which experts in the lower half-plane always target goal g_l . RTC uses the BC loss as reconstruction loss $\mathcal{L}_{\text{rec}}(\tau) = -\log \pi_\theta(a_t | s_t, \hat{g}_e)$

TABLE I: Averages and standard deviation over 20 training runs on *WOMD*. The best two values are highlighted.

	Collision rate (%) ↓	Off-road time (%) ↓	MinADE (m) ↓	Curvature JSD ($\times 10^{-3}$) ↓	Progress JSD ($\times 10^{-3}$) ↓
Data Distribution	1.16	0.68	-	-	-
MGAIL	5.39 ± 0.68	0.89 ± 0.12	1.34 ± 0.08	1.32 ± 1.48	3.81 ± 1.29
Symphony	6.39 ± 0.95	0.90 ± 0.06	1.40 ± 0.12	0.97 ± 0.62	6.44 ± 5.25
InfoMGAIL - C	5.21 ± 0.37	0.89 ± 0.14	1.29 ± 0.07	1.24 ± 0.93	4.40 ± 1.47
InfoMGAIL - D	4.82 ± 0.29	0.84 ± 0.10	1.35 ± 0.11	0.77 ± 0.44	4.01 ± 1.45
NaiveHierarchy	35.08 ± 0.44	1.83 ± 0.42	1.12 ± 0.01	1.76 ± 2.05	2.54 ± 0.63
RTC - C	4.23 ± 0.16	0.68 ± 0.04	1.15 ± 0.10	0.43 ± 0.06	2.17 ± 0.65
RTC - D	4.21 ± 0.24	0.74 ± 0.06	1.12 ± 0.10	0.89 ± 0.66	2.56 ± 0.54

and continuous types. All policies use a bimodal Gaussian mixture model as action distribution.

This experiment combines agent-internal decisions (which goal to approach) with external stochasticity (goal starting positions and movement directions). Task performance is measured as the number of steps for which the agent is within $\delta = 0.1$ distance of one of the goals (higher is better). Distributional realism is measured as the divergence between the empirical distributions, $\text{JSD}(p_{\text{agent}}(h_s) \| p_{\text{expert}}(h_s))$ (lower is better) where we take $h_s = \text{sign}(y_T)$ of the final agent position $[x_T, y_T]$ to indicate which goal was approached. Our aim is to improve distributional realism while maintaining or improving task performance.

Figure 5 shows that MGAIL improves task performance compared to BC. Our method, RTC, improves it further, possibly because given a type, the required action distribution is unimodal. Importantly, RTC substantially improves distributional realism, achieving lower JSD values. The bias introduced by the information-theoretic loss in InfoMGAIL reduces task performance without improving distributional realism. Lastly, NaiveHierarchy achieves excellent distributional realism through the learned hierarchy but suffers reduced task performance due to conditional type shift.

B. Waymo Open Motion Dataset (WOMD)

To evaluate RTC on a complex environment we use the *Waymo Open Motion Dataset* [20] consisting of 487K segments of real world driving behaviour, each 9s long at 10Hz. We follow [9] by controlling agents at 3.3Hz and replaying uncontrolled agents from logs. Distributionally realistic agents are critical for driving simulations, for example for estimating safety metrics. Diverse intents and driving styles cause the data to be highly multimodal. External stochasticity is induced through the unpredictable behaviour of other cars, cyclists and pedestrians. We use $\mathcal{L}_{\text{rec}}(\tau, \hat{\tau}) = \sum_t^T \mathcal{L}_{\text{Huber}}(s_t, \hat{s}_t)$ where $\mathcal{L}_{\text{Huber}}$ is the average Huber loss of the four vehicle bounding box corners.

The percentage of segments with collisions and time spent off-road are proxy metrics for task performance and realism. Mode coverage is measured by the minimum average displacement error, $\text{minADE} = \mathbb{E}_{\tau \sim \mathcal{D}, \{\hat{\tau}_i\}_i^K \sim \pi_{\theta}} \left[\min_{\hat{\tau}_i} \frac{1}{T} \sum_{t=1}^T \delta(s_t, \hat{s}_{i,t}) \right]$, where δ is the Euclidean distance between agent positions and we find the minimum over $K = 16$ rollouts (hierarchical methods use

K independently sampled types). Lower *minADE* implies better mode coverage, but does not directly measure the relative frequency of modes, e.g., low probability modes may be overrepresented. To measure distribution matching in driving intent, we use the *Curvature JSD* [9]: in lane branching regions, such as intersections, it maps trajectories to the nearest lane and extracts its curvature as feature h_{cur} . The driving style distribution is measured through the progress feature $h_{\text{style}} = \delta(\hat{s}_0, \hat{s}_T)$. To compute $\text{JSD}(p_{\text{agent}}(h_{\text{cur}/\text{style}}) \| p_{\text{expert}}(h_{\text{cur}/\text{style}}))$, the value of $h_{\text{cur}/\text{style}}$ is discretized into 100 equisized bins.

Results are provided in table I. Both versions of RTC improve task performance (collisions and off-road events) and distributional realism metrics (minADE and divergences) compared to the flat MGAIL baseline and previous hierarchical approaches (Symphony, InfoMGAIL, NaiveHierarchy). Both type representations, RTC-C and RTC-D, perform similarly, showing robustness of RTC to different implementations.

MGAIL achieves good task performance, but is outperformed by RTC due to the use of hierarchy. On the other hand, Symphony, using lane segment goals to capture driving intent, consequently improves on the *Curvature JSD* distributional realism metric, but not on *Progress JSD* which measures driving style, not intent. In contrast, RTC improves on both distributional realism metrics since the fully learned type is more expressive. The information-theoretic loss in InfoMGAIL improves distributional realism on some metrics, but is less effective than RTC: while the additional InfoMGAIL loss ensures that the type contains some information, it does not require this information to be useful, unlike in an autoencoder framework.

Lastly, the advantage of RTC in achieving *both* good task performance and distributional realism becomes clearest by comparing it to NaiveHierarchy. While NaiveHierarchy achieves some improvements in distributional realism, it has nearly an order of magnitude more collisions. This is a consequence of the challenges discussed in section III: At training time, the inferred type contains too much information, for example when to break or start driving. At test time, because this information is sampled independently to what is actually happening in the environment, the agent behaves incorrectly and collides with other road users.

VII. CONCLUSIONS, LIMITATIONS, AND FUTURE WORK

This paper identified new challenges in learning hierarchical policies from demonstration to capture multimodal trajectory distributions in stochastic environments. We expressed them as *conditional type shifts* in the hierarchical policy. We proposed *Robust Type Conditioning* (RTC) to eliminate these distribution shifts and showed improved distributional realism while maintaining or improving task performance on two stochastic environments, including the Waymo Open Motion Dataset [20]. Future work will address *conditional* distributional realism by not only matching the marginal distribution $p(\tau)$, but the conditional distribution $p(\tau|\xi)$ under a specific realization of the environment. For example, drivers might change their intent based on the current traffic situation or players might adapt their strategy as the game unfolds. Achieving such conditional distributional realism will also require new models and metrics.

REFERENCES

- [1] D. Amodi, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in ai safety," *arXiv preprint arXiv:1606.06565*, 2016.
- [2] D. Hadfield-Menell, S. Milli, P. Abbeel, S. J. Russell, and A. Dragan, "Inverse reward design," *Advances in neural information processing systems*, vol. 30, 2017.
- [3] J. Fu, K. Luo, and S. Levine, "Learning robust rewards with adversarial inverse reinforcement learning," in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=rkHywl-A->
- [4] T. Everitt, M. Hutter, R. Kumar, and V. Krakovna, "Reward tampering problems and solutions in reinforcement learning: A causal influence diagram perspective," *Synthese*, vol. 198, no. 27, pp. 6435–6467, 2021.
- [5] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine, "Learning complex dexterous manipulation with deep reinforcement learning and demonstrations," *arXiv preprint arXiv:1709.10087*, 2017.
- [6] Y. Zhu, Z. Wang, J. Merel, A. Rusu, T. Erez, S. Cabi, S. Tunyasuvunakool, J. Kramár, R. Hadsell, N. de Freitas *et al.*, "Reinforcement and imitation learning for diverse visuomotor skills," *arXiv preprint arXiv:1802.09564*, 2018.
- [7] J. D. Farmer and D. Foley, "The economy needs agent-based modelling," *Nature*, vol. 460, no. 7256, pp. 685–686, 2009.
- [8] S. Suo, S. Regalado, S. Casas, and R. Urtasun, "TrafficSim: Learning to simulate realistic multi-agent behaviors," in *ICCV*, 2021.
- [9] M. Igl, D. Kim, A. Kuefler, P. Mougin, P. Shah, K. Shiarlis, D. Anguelov, M. Palatucci, B. White, and S. Whiteson, "Symphony: Learning realistic and diverse agents for autonomous driving simulation," in *ICRA*, 2022.
- [10] A. Grover, M. Al-Shedivat, J. Gupta, Y. Burda, and H. Edwards, "Learning policy representations in multiagent systems," in *International conference on machine learning*. PMLR, 2018, pp. 1802–1811.
- [11] Y. Liang, C. Guo, Z. Ding, and H. Hua, "Agent-based modeling in electricity market using deep deterministic policy gradient algorithm," *IEEE Transactions on Power Systems*, vol. 35, no. 6, 2020.
- [12] Z. Wang, J. S. Merel, S. E. Reed, N. de Freitas, G. Wayne, and N. Heess, "Robust imitation of diverse behaviors," *NeurIPS*, 2017.
- [13] M. Lucic, K. Kurach, M. Michalski, S. Gelly, and O. Bousquet, "Are gans created equal? a large-scale study," *NeurIPS*, 2018.
- [14] A. Creswell, T. White, V. Dumoulin, K. Arulkumaran, B. Sengupta, and A. A. Bharath, "Generative adversarial networks: An overview," *IEEE signal processing magazine*, vol. 35, no. 1, pp. 53–65, 2018.
- [15] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," *Communications of the ACM*, vol. 63, no. 11, pp. 139–144, 2020.
- [16] J. Ho and S. Ermon, "Generative adversarial imitation learning," *NeurIPS*, 2016.
- [17] N. Baram, O. Anschel, and S. Mannor, "Model-based adversarial imitation learning," *arXiv preprint arXiv:1612.02179*, 2016.
- [18] D. Xu, Y. Chen, B. Ivanovic, and M. Pavone, "Bits: Bi-level imitation for traffic simulation," *arXiv preprint arXiv:2208.12403*, 2022.
- [19] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," *JMLR Workshop and Conference Proceedings*, 2011.
- [20] S. Ettinger, S. Cheng, B. Caine, C. Liu, H. Zhao, S. Pradhan, Y. Chai, B. Sapp, C. Qi, Y. Zhou, Z. Yang, A. Chouard, P. Sun, J. Ngiam, V. Vasudevan, A. McCauley, J. Shlens, and D. Anguelov, "Large scale interactive motion forecasting for autonomous driving : The waymo open motion dataset," *CoRR*, 2021.
- [21] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *ICLR*, 2014.
- [22] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet, "Learning latent plans from play," in *Conference on robot learning*. PMLR, 2020, pp. 1113–1132.
- [23] P. De Haan, D. Jayaraman, and S. Levine, "Causal confusion in imitation learning," *NeurIPS*, 2019.
- [24] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *ICML*, 2015.
- [25] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.
- [26] R. S. Sutton, D. Precup, and S. Singh, "Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning," *Artificial intelligence*, vol. 112, no. 1-2, pp. 181–211, 1999.
- [27] P.-L. Bacon, J. Harb, and D. Precup, "The option-critic architecture," in *AAAI*, 2017.
- [28] A. S. Vezhnevets, S. Osindero, T. Schaul, N. Heess, M. Jaderberg, D. Silver, and K. Kavukcuoglu, "Feudal networks for hierarchical reinforcement learning," in *ICML*, 2017.
- [29] O. Nachum, S. Gu, H. Lee, and S. Levine, "Near-optimal representation learning for hierarchical reinforcement learning," in *ICLR*, 2019.
- [30] M. Igl, A. Gambardella, J. He, N. Nardelli, N. Siddharth, W. Böhm, and S. Whiteson, "Multitask soft option learning," in *UCA*, 2020.
- [31] S. Krishnan, R. Fox, I. Stoica, and K. Goldberg, "Ddco: Discovery of deep continuous options for robot learning from demonstrations," in *Conference on robot learning*. PMLR, 2017, pp. 418–437.
- [32] H. Le, N. Jiang, A. Agarwal, M. Dudik, Y. Yue, and H. Daumé III, "Hierarchical imitation and reinforcement learning," in *ICML*, 2018.
- [33] K. Shiarlis, M. Wulfmeier, S. Salter, S. Whiteson, and I. Posner, "Taco: Learning task decomposition via temporal alignment for control," in *ICML*, 2018.
- [34] J. Merel, Y. Tassa, D. TB, S. Srinivasan, J. Lemmon, Z. Wang, G. Wayne, and N. Heess, "Learning human behaviors from motion capture by adversarial imitation," *arXiv:1707.02201*, 2017.
- [35] C. Fei, B. Wang, Y. Zhuang, Z. Zhuang, J. Hao, H. Zhang, X. Ji, and W. Liu, "Triple-gail: a multi-modal imitation learning framework with generative adversarial nets," *arXiv preprint arXiv:2005.10622*, 2020.
- [36] A. Tamar, K. Rohanimanesh, Y. Chow, C. Vigorito, B. Goodrich, M. Kahane, and D. Pridmore, "Imitation learning from visual data with multiple intentions," in *ICLR*, 2018.
- [37] S. Khandelwal, W. Qi, J. Singh, A. Hartnett, and D. Ramanan, "What-if motion prediction for autonomous driving," *arXiv preprint arXiv:2008.10587*, 2020.
- [38] Y. Ding, C. Florensa, P. Abbeel, and M. Phielipp, "Goal-conditioned imitation learning," *NeurIPS*, 2019.
- [39] A. Pashevich, C. Schmid, and C. Sun, "Episodic transformer for vision-and-language navigation," in *ICCV*, 2021.
- [40] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev *et al.*, "Grandmaster level in starcraft ii using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.
- [41] Y. Li, J. Song, and S. Ermon, "Infogail: Interpretable imitation learning from visual demonstrations," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [42] K. Hausman, Y. Chebotar, S. Schaal, G. Sukhatme, and J. J. Lim, "Multi-modal imitation learning from unstructured demonstrations using generative adversarial nets," in *NeurIPS*, 2017.
- [43] B. Eysenbach, A. Gupta, J. Ibarz, and S. Levine, "Diversity is all you need: Learning skills without a reward function," in *ICLR*, 2019.
- [44] D. Pathak, D. Gandhi, and A. Gupta, "Self-supervised exploration via disagreement," in *ICML*, 2019.